Iterables and more on functions

What a `for` loop can walk, `iter` and `next` underneath, functions that take any iterable, `zip`, and tuples.

THE QUESTION

What can a `for` loop walk through, and how do several values go into and out of one function?

THE INTUITION

Anything a `for` loop can walk is an iterable: a list, a string, a range, a tuple. A tuple is a fixed row of values; it lets a function return several values and one line unpack them into names.

KEY TAKEAWAY

An iterator is used up after one walk. Unpack with one name per value. Never give a parameter a list as its default.

`for x in items:` → `it = iter(items)` → `x = next(it)` → run the body → again, until StopIteration → carry on after the loop

1 A `for` loop can walk any iterable

for v in X: gives v these values, one per pass

`[10, 20, 30]`	a list	`10` `20` `30`
`"hi!"`	a string	`'h'` `'i'` `'!'`
`range(2, 5)`	a range	`2` `3` `4`
`(1, "a")`	a tuple	`1` `'a'`
`{"x": 1, "y": 2}`	a dict: its keys	`'x'` `'y'`
`42`	an int	`TypeError: 'int' object is not iterable`

`list("ab c")` → `['a', 'b', ' ', 'c']` `list(range(2, 5))` → `[2, 3, 4]`

An **iterable** is anything a `for` loop can walk, one item per pass. A **tuple** is in panel 6; a **dictionary** gives its keys (the dictionaries sheet). An int is one value, so walking it is a **TypeError**. `list(x)` keeps the items of any iterable.

2 `iter` and `next`: what `for` does underneath

at the >>> prompt

```
>>> nums = [10, 20, 30]
>>> it = iter(nums)
>>> next(it)
10
>>> next(it)
20
>>> next(it)
30
>>> next(it)
StopIteration
```

the iterator keeps its place in the list

next 1	10	20	30	→ 10
next 2	10	20	30	→ 20
next 3	10	20	30	→ 30
next 4	10	20	30	→ StopIteration

triangle: where the iterator stands

for x in nums: does this for you

`it = iter(nums)` → `x = next(it); run the body` → `StopIteration: leave`

`iter(x)` gives an **iterator**, an object that remembers how far the walk has got. `next(it)` hands over one item and moves on; past the end it raises **StopIteration**. A `for` loop calls `iter` once and `next` before each pass, and stops quietly at StopIteration.

3 An iterator is used up after one walk

one session at >>>

```
>>> it = iter([1, 2, 3])
>>> list(it)
[1, 2, 3]
>>> list(it)
[]
>>> r = range(3)
>>> sum(r), sum(r)
(3, 3)
>>> nums = [1, 2, 3]
>>> e = enumerate(nums)
>>> nums[0] = 99
>>> list(e)
[(0, 99), (1, 2), (2, 3)]
```

with a = [1, 2]; tried for real

X	walk again	X[0]	len(X)
[1, 2]	yes	yes	yes
"ab"	yes	yes	yes
(1, 2)	yes	yes	yes
range(2)	yes	yes	yes
iter(a)	no	no	no
zip(a, a)	no	no	no
enumerate(a)	no	no	no
map(abs, a)	no	no	no

purple: iterators, used up after one walk

An iterator hands out each item once, so a second `list(it)` gives `[]`. `zip`, `enumerate` and `map` (the functions are values sheet) give iterators too. They are **lazy**: an item is worked out only when asked for, so `list(e)` sees the 99. A list is **eager**: its items exist from the start.

4 Functions that take any iterable

X is

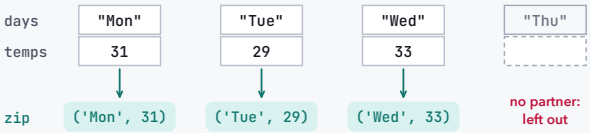
	[3, 1, 2]	"cab"	range(1, 4)
len(X)	3	3	3
sum(X)	6	TypeError	6
max(X)	3	'c'	3
min(X)	1	'a'	1
sorted(X)	[1, 2, 3]	['a', 'b', 'c']	[1, 2, 3]
list(X)	[3, 1, 2]	['c', 'a', 'b']	[1, 2, 3]
"-".join(X)	TypeError	'c-a-b'	TypeError

sum adds numbers only; join glues strings only (the strings sheet)

`sorted([3, 1, 2], reverse=True)` → `[3, 2, 1]` `max("hello")` → `'o'`

These take any iterable, and `sep.join` any iterable of strings. `sorted` always gives a list. `max` and `min` of a string compare character codes (the strings sheet). `len` fails on an iterator, which does not know its length until it is walked.

5 `zip` pairs items side by side




```
1 days = ["Mon", "Tue", "Wed", "Thu"]
2 temps = [31, 29, 33]
3 for d, t in zip(days, temps):
4     print(d, t)
```

prints
```
Mon 31
Tue 29
Wed 33
```

`list(zip(days, temps))` → `[('Mon', 31), ('Tue', 29), ('Wed', 33)]`

`zip(a, b)` hands out tuples of the first items, the second items and so on, and stops silently at the end of the shorter input: Thu is left out. The `for` line **unpacks** each pair. `enumerate` (the for loops sheet) is `zip` with a counter.

6 A tuple is a list that cannot change

a tuple reads like a list

```
>>> t = (4, 7, 9)
>>> t[1]
7
>>> t[1:]
(7, 9)
>>> t[0] = 5
TypeError: 'tuple' object does
not support item assignment
```

commas pack; names unpack

```
>>> p = 4, 5
>>> p
(4, 5)
>>> x, y = p
>>> y
5
>>> type((7,))
<class 'tuple'>
```

the notebook: the for line unpacks each pair

```
1 for x, y in [(1, 2), (3, 4), (5, 6)]:
2     print(x + y)
```

prints `3 7 11`

pass	item	x	y	x + y
1	(1, 2)	1	2	3
2	(3, 4)	3	4	7
3	(5, 6)	5	6	11

A **tuple** is an ordered row of values in round brackets. Indexing, slicing, `len` and `for` work as on a list, but it is **immutable** (the Python memory sheet). Commas make a tuple: `p = 4, 5` **packs**; `(7,)` has one item, `(7)` is just 7. `x, y = p` **unpacks** it.

Several values in, several values out

The one-line swap, returning a tuple, default and keyword arguments, and the default list that is made only once.

7 `a, b = b, a` swaps in one line

one line: the right side first

```
1 a = 3
2 b = 8
3 a, b = b, a
4 print(a, b)
```

prints `8 3`

step	right side	a	b
after lines 1 and 2		3	8
line 3: work out b, a	(8, 3)	3	8
line 3: unpack into a, b		8	3

the tuple holds the old values, so none is lost

two lines: the 3 is lost

line	code	a	b
1	a = 3	3	
2	b = 8	3	8
3	a = b	8	8
4	b = a	8	8

prints `8 8`

fix without a tuple:

```
tmp = a
a = b
b = tmp
```

prints `8 3`

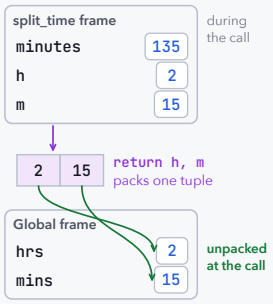
Python works out the right side first, packing the old values into `(8, 3)`, then unpacks into `a` and `b`. With two lines, `a = b` throws the 3 away before `b = a` could copy it. A third name also works.

8 Return several values as one tuple

```
1 def split_time(minutes):
2     h = minutes // 60
3     m = minutes % 60
4     return h, m
5
6 hrs, mins = split_time(135)
7 print(hrs, "h", mins, "min")
```

prints `2 h 15 min`

```
split_time(135) → (2, 15)
type(split_time(135)) → <class 'tuple'>
divmod(135, 60) → (2, 15)
```



`return h, m` packs two values into one tuple, which the caller unpacks. The lecture's rule: no `return` gives `None` (the functions sheet), one value gives that value, several give a tuple, as `divmod` does.

9 Default and keyword arguments

```
1 def greet(name, greeting="Hello", mark="!"):
2     return greeting + ", " + name + mark
```

greeting and mark have defaults

call	name	greeting	mark	returns
greet("Alice")	"Alice"	"Hello"	"!"	'Hello, Alice!'
greet("Bob", "Hi")	"Bob"	"Hi"	"!"	'Hi, Bob!'
greet("Carol", mark="?")	"Carol"	"Hello"	"?"	'Hello, Carol?'
greet(mark=".", name="Dan")	"Dan"	"Hello"	"."	'Hello, Dan.'
greet()				TypeError

by position · by name (keyword) · the default

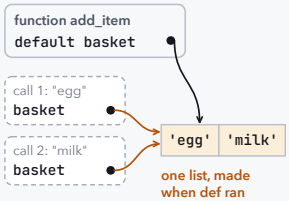
A **default argument** is used when a call leaves that parameter out. A **keyword argument** names its parameter, so its place does not matter, as with `print`'s `sep` and `end`. **Positional** arguments fill parameters from the left and come before keyword ones. In a `def`, parameters with defaults come last.

10 A default is made once, when `def` runs

the trap

```
1 def add_item(item, basket=[]):
2     basket.append(item)
3     return basket
4
5 print(add_item("egg"))
6 print(add_item("milk"))
```

prints `['egg']`
`['egg', 'milk']`



the fix: default None, a new list inside

```
1 def add_item(item, basket=None):
2     if basket is None:
3         basket = []
4     basket.append(item)
5     return basket
```

prints `['egg'] then ['milk']`

add_item.__defaults__ after the calls
`(['egg', 'milk'],)`

Python works out a default once, when it runs the `def` line, and keeps it with the function, so calls that leave `basket` out share one list. Use `None` instead. Likewise `n = 5, def f(x=n), n = 7: f()` gives 5.

11 Common mistakes: what you wrote, what Python says or prints, the fix

you wrote	the last line Python prints, or the wrong output; why, and the fix
`total = sum(5, 6)`	`TypeError: 'int' object is not iterable` `sum` takes one iterable: `sum([5, 6])` gives 11

`pairs = zip("ab", [1, 2])`
`print(list(pairs))`
`print(list(pairs))`
	`prints [('a', 1), ('b', 2)] then []`
	a `zip` is used up by the first `list`; keep `pairs = list(zip(...))` to walk it again

`x, y = (1, 2, 3)`	`ValueError: too many values to unpack (expected 2)` one name per value: `x, y, z = (1, 2, 3)`; too few values say "not enough values"
`greet(greeting="Hi", "Alice")`	`SyntaxError: positional argument follows keyword argument` positional first: `greet("Alice", greeting="Hi")` gives `Hi, Alice!`

12 Try it (answers in green)

1. `list(zip("abc", [1, 2]))`?
   `[('a', 1), ('b', 2)]`

2. `it = iter("abc")`, `next(it)`, then `list(it)`?
   `['b', 'c']`

3. `a, b, c = "xyz"`, then `c + a`?
   `'zx'`

4. `sorted("bca")`, `max("hello")`?
   `['a', 'b', 'c']`, `o`

5. `def p(x, n=2): return x ** n`: `p(3)`, `p(3, 3)`, `p(n=1, x=5)`?
   `9, 27, 5`

6. `mm(xs)` returns `min(xs), max(xs)`. `lo, hi = mm(range(3, 8))`?
   `3, 7`

iterables feed `for` · an iterator is used up · commas pack, names unpack · positional before keyword

Source: the second functions notebook and the next class's notebook and notes (iterables, tuples, eager versus lazy), nipunbatra.github.io/comp22; `zip`, `iter`, `next` and the argument rules complete the topic. Outputs from Python 3.11.11. Next: the functions are values sheet.